COMPUTER ENGINEERING

QUEZON CITY

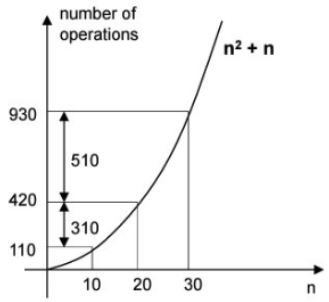
COURSE CODE/TITLE:	CPE010	**SECTION:**	CPE12S4
DATE PERFORMED:	Oct 7, 2026	**DATE SUBMITTED:**	Oct 7, 2026
NAME:	Cardenas and De Vera	**INSTRUCTOR:**	Engr. Jimlord Quejado

ACTIVITY NO. 11.1 BASIC ALGORITHM ANALYSIS

1. PROCEDURE OUTPUT

ILO A: Runtime of algorithms using theoretical analysis

```
bool diff(int *x, int *y){
    for(int i = 0; i < y.length; i++){
        if(search(x, y[i])!= - 1){
            return false;
        }
    }
    return true;
}
```



n (size of array)	number of operations
10	110
20	420
30	930

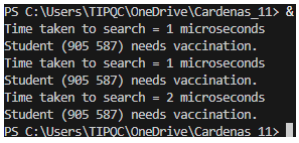
ANALYSIS: *As the array size increases, the number of operations also increases. The table shows 110 operations for n = 10, 420 for n = 20, and 930 for n = 30. The jump from 10 to 20 is 310 operations, while the jump from 20 to 30 is 510, so the growth gets bigger each time. This is why the graph forms a curve instead of a straight line. The function has a loop that calls* `search` *for every element of* `y`*, and each search may scan the whole array* `x`*. This gives about n × n operations, which matches the formula n² + n. Therefore, the algorithm becomes more costly as the input grows, and its worst-case complexity is O(n²).*

ILO B outputs (De Vera outputs)

Input Size (N)	Execution Speed	Screenshot	Observations
1,000 population size	0 microseconds		The output was displayed as soon as possible.
10,000 population size	0 microseconds	Time taken to search = 0 microseconds Student (196 3) needs vaccination. Time taken to search = 0 microseconds Student (196 3) needs vaccination. Time taken to search = 0 microseconds Student (196 3) needs vaccination.	Searching for the student was extremely fast, 0ms search pace for ten thousands of population size.
100,000 population size	0 microseconds		For a hundred thousand class size, a search of 0ms may indicate that student is not on the list or simply the searching time (space complexity of the computer)

			is fast

ILO B outputs (Cardenas outputs)

Input Size (N)	Execution Speed	Screenshot	Observations
1000 population size	1 microseconds		It finished within a 1 microseconds on the small dataset. This result serves as a baseline for comparing other population sizes.
10,000 population size	1 microseconds		Although the population size grew multiple sizes the execution time stayed 1 microseconds then there is no measurable slowdown. The same student was found and the same output was displayed. This shows that the search time did not increase in proportion to the input size, which would not be the case for a linear search that checks records one by one.
100,000 population size	2 microseconds		With the one hundred times larger in the first test because the time increased from 1 to 2 microseconds. This is a negligible increase compared with the huge growth in data. A linear search would have taken roughly a hundred times longer, so this result indicates the algorithm scales very well. The small change may also come from timer resolution or normal system activity rather than from the algorithm itself.

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

Problem 1: Consider a program that returns true if the elements in an array are unique.

CODE with OUTPUT:

HOA11_Supple.cpp > test(int)

```cpp
#include <iostream>
#include <vector>
#include <chrono>

bool Unique(std::vector<int>& A) {
    int n = A.size();

    for (int i = 0; i < n - 1; i++) {
        for (int j = i + 1; j < n; j++) {
            if (A[i] == A[j])
                return false;
        }
    } <- #8-13 for (int i = 0; i < n - 1; i++)

    return true;
} <- #5-16 bool Unique(std::vector<int>& A)

void test(int size) {
    std::vector<int> A(size);

    // Create an array with unique elements
    for (int i = 0; i < size; i++)
        A[i] = i;

    std::chrono::steady_clock::time_point begin =
        std::chrono::steady_clock::now();

    bool result = Unique(A);

    std::chrono::steady_clock::time_point end =
        std::chrono::steady_clock::now();
```

```cpp
    std::cout << "Input Size (N): " << size << std::endl;
    std::cout << "Time taken: "<< std::chrono::duration_cast<std::chrono::milliseconds> (end - begin).count() << " milliseconds" << std::endl;

    if (result)
        std::cout << "All elements are unique." << std::endl;
    else
        std::cout << "Duplicate elements found." << std::endl;

    std::cout << std::endl;
} <- #18-42 void test(int size)
```

```cpp
int main() {

    test(1000);
    test(10000);
    test(100000);

    return 0;
} <- #44-51 int main()
```

1st run:

```
Input Size (N): 1000
Time taken: 4 milliseconds
All elements are unique.

Input Size (N): 10000
Time taken: 386 milliseconds
All elements are unique.

Input Size (N): 100000
Time taken: 39983 milliseconds
All elements are unique.
```

Analysis:

- *The code program first checks the array list generated by itself and checks if there's any duplication on the list. It then outputs **true** if elements appear to be unique (no duplication seen) or **false** when there are at least two the same elements generated.*
- *The array size is represented by **n** as the number of elements. The whole bool function depends on this line of code.*
- *The for loop chooses the first element that we needed to compare first. Then the second for loop compares A[i] with the elements after it.*
- *When the program finishes up to the last incrementation, when none of the array seems to have a duplication, the code will return true and will indicate that the array elements are unique.*
- *The main function runs three tests for 1000, 10000, and 100000 population size. Each test determines how long each execution will different from each other as the array size grows larger.*
- *The time complexity of this program code is $O(n^2)$ because the population size grows in a quadratic algorithm and the running time gets slower and slower, as seen in the results.*

2nd Run:

```
Input Size (N): 1000
Time taken: 3 milliseconds
All elements are unique.

Input Size (N): 10000
Time taken: 394 milliseconds
All elements are unique.

Input Size (N): 100000
Time taken: 40063 milliseconds
All elements are unique.
```

Problem 2: Consider another program that raises a number x to an arbitrary nonnegative integer, n.

CODE with OUTPUT:

HOA11_SuppleB.cpp > main()

```cpp
#include <iostream>
#include <chrono>

long long rpower(int x, int n) {
    if (n == 0)
        return 1;
    else
        return x * rpower(x, n - 1);
} <- #4-9 long long rpower(int x, int n)

long long brpower(int x, int n) {
    if (n == 0)
        return 1;

    if (n % 2 != 0) {
        long long y = brpower(x, (n - 1) / 2);
        return x * y * y;
    }

    long long y = brpower(x, n / 2);
    return y * y;
} <- #11-22 long long brpower(int x, int n)

```





Analysis:

- *The program requires us to show two different recursive methods: (1) regular recursive power and (2) binary recursive power. The code is a function to measure how long each method can run depending on the population size set, which are 1000, 10000, 100000.*
- *The regular recursive power works by subtracting 1 to the array size every time. For example, in the code, we used 1000 as the population size. So the function subtracts 1, now we have 999 and continues to do so until it reaches 0. This is a linear runtime which means it is an O(n).*
- *Next is the binary recursive power which divides the array into 2 by finding its midpoint. Its time complexity is O(log n).*
- *Between the two it is obvious that the BRPower is way faster than the RPower since it divides its array into halves, lowering the running time of the code.*
- *As for the output, despite writing a test_rpower and test_brpower with a population size of 1000, 10000, and 100000, we only got 1000 and 10000 outputs then the code hangs/stops and returns you to the terminal/command prompt.*
- *We deduced that as the program pushes up*

```
int main() {

    test_rpower(2, 1000);
    test_rpower(2, 10000);
    test_rpower(2, 100000);

    test_brpower(2, 1000);
    test_brpower(2, 10000);
    test_brpower(2, 100000);

    return 0;
} <- #58-69 int main()
```

```
rpower
Input Size (N): 1000
Time taken: 28 microseconds

rpower
Input Size (N): 10000
Time taken: 342 microseconds
```

```
rpower
Input Size (N): 1000
Time taken: 41 microseconds

rpower
Time taken: 41 microseconds

rpower
Input Size (N): 10000
Time taken: 605 microseconds
Input Size (N): 10000
Time taken: 605 microseconds
```

Running for 100000 population size:
<u>(no output)</u>

to 100,000 recursive calls, the program cannot call a very large stack and makes the program run a stack overflow or otherwise terminate abnormally before it can print the result.

- *This creates a very deep recursion and can exceed the available call stack, causing the program to terminate before producing the 100,000 result.*

CONCLUSION:

- *In this hands-on activity, I think we are put into the test on analysing the different methods of algorithm analysis and methods of efficiency. Especially that we have tackled and done the regular recursive power and binary recursive power in the supplementary activity. We have to differentiate which recursive power takes an efficient way on calling a function that does not take a long time on running the program. Between the two, the binary recursive power is obviously more efficient than the regular recursive power because it divides the input size by half in every recursive call, while the regular recursive power only subtracts 1. It is more advantageous than calling the program a thousand times individually.*
- *In this hands-on activity, we were able to apply algorithm analysis by comparing the theoretical complexity with the actual running times we measured. We observed that algorithms with nested loops, like the duplicate checker, slow down dramatically as the input grows, while a fast search or a divide-and-conquer approach stays efficient. In the supplementary activity, the binary recursive power proved to be more efficient than the regular recursive power because it cuts the problem in half each time, while the regular one only reduces it by 1 and can overflow the stack at very large inputs. This showed us that writing code that works is not enough; it must also be efficient enough to handle large inputs.*

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: **Google Overview**

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: **Concept clarification and explanation.**

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use:

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.


T.I.P
TECHNOLOGICAL INSTITUTE OF THE PHILIPPINES
COMPUTER ENGINEERING
QUEZON CITY